DVWA Security Level Comparison – SQL Injection & Cross-Site Scripting

Objective

The objective of this project is to compare how DVWA behaves at different security levels (Low, Medium, High). Two common web vulnerabilities – SQL Injection and Cross-Site Scripting (XSS), are used to identify the defense mechanisms implemented at each level.

Environment Setup:

- **Platform:** Kali Linux
- **Application:** DVWA (Damn Vulnerable Web App)
- **Start DVWA:**

  sudo dvwa-start
- **Login Credentials:**

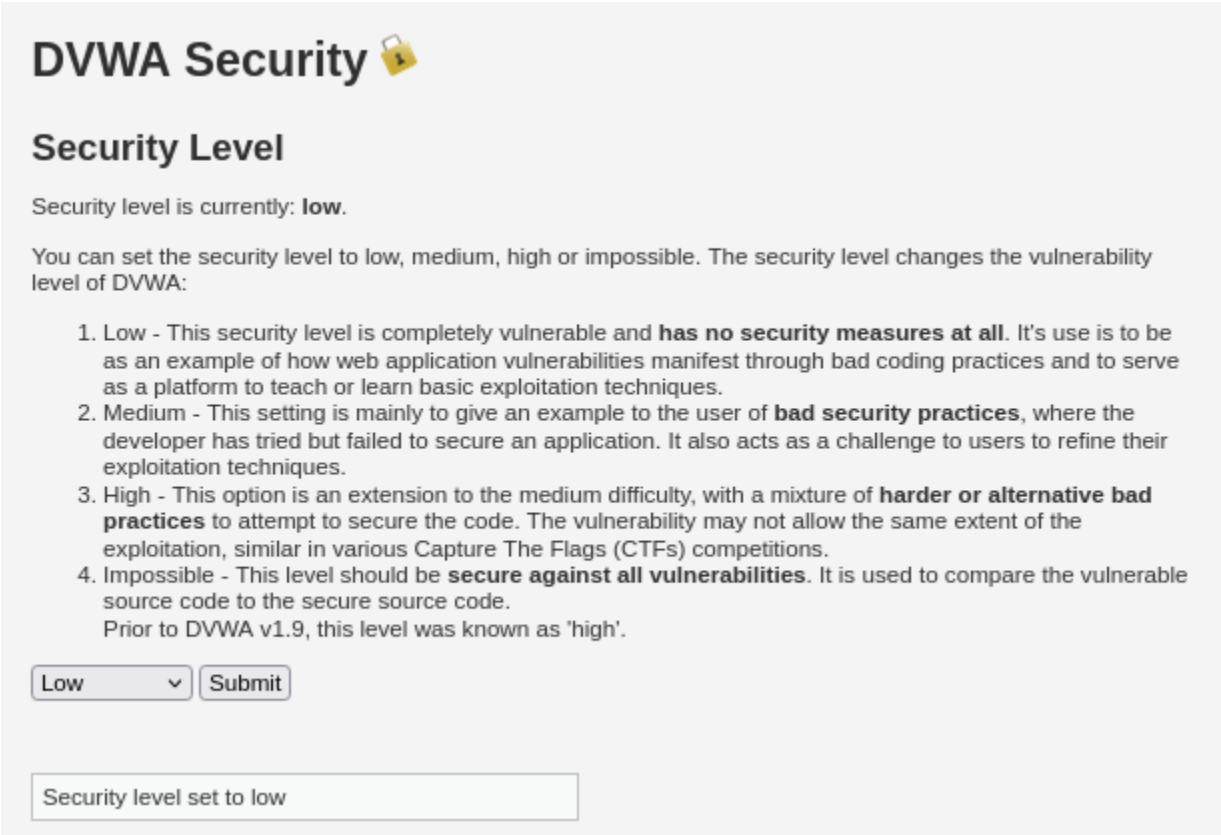
  1. Username: admin
  2. Password: password

Methodology:

1. SQL Injection

- **Low Security**

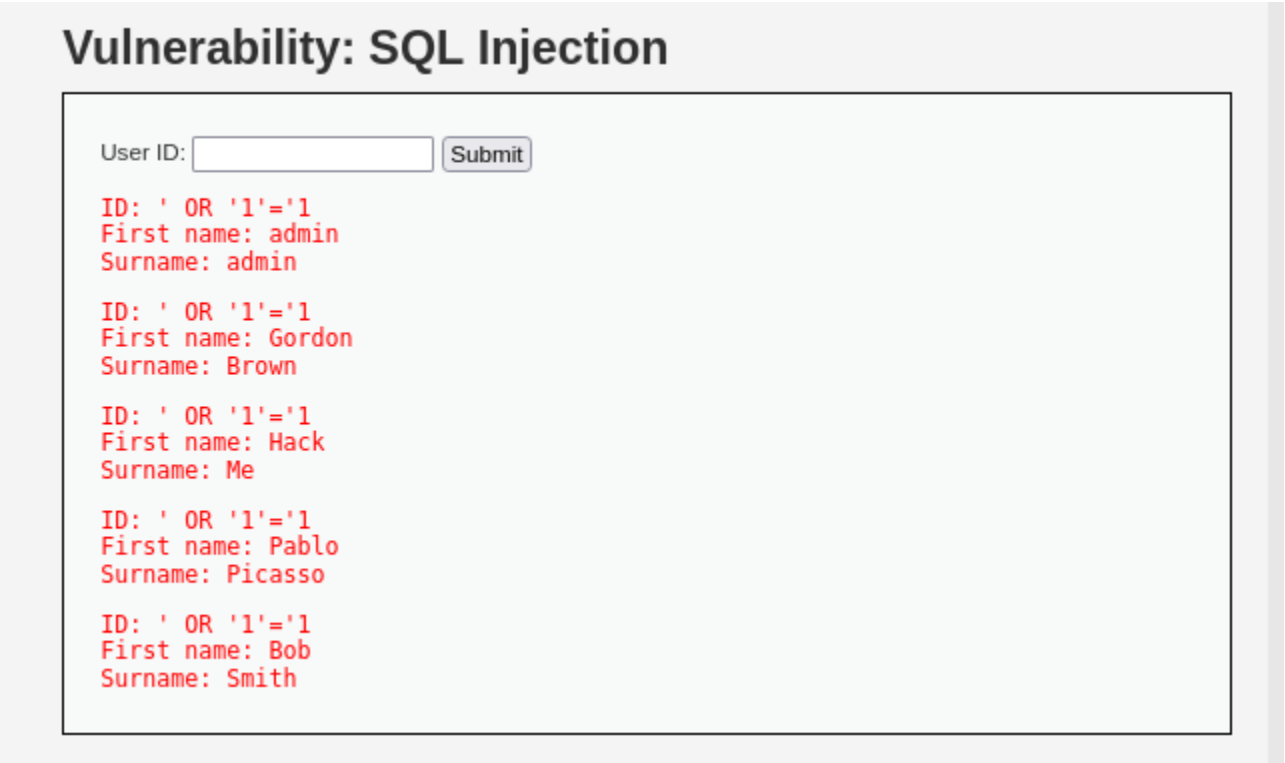
  → DVWA Security → Set Security to **Low**



- Open **SQL Injection** page.

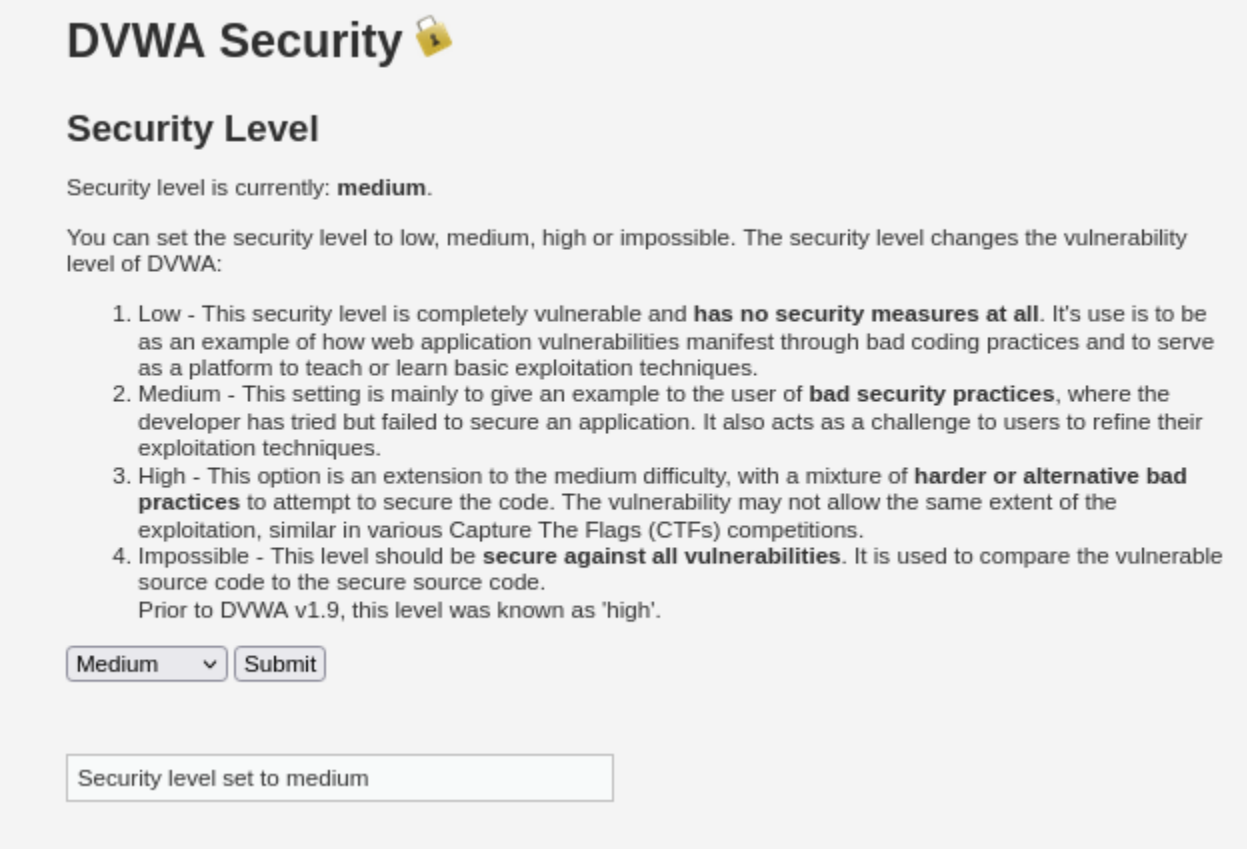
Payload: **1’ OR ‘1’=’1**

Result:

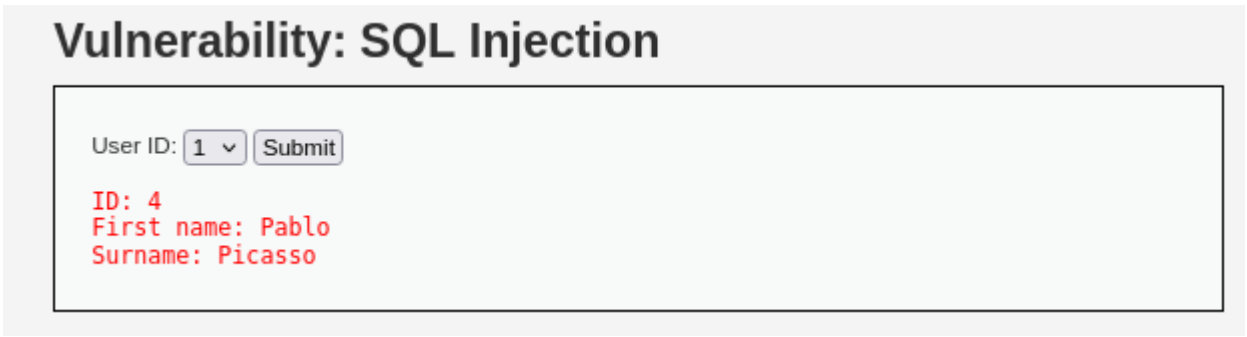


- **Medium Security**

  → Changed Security level to **Medium.**



Result:



Http Capture:

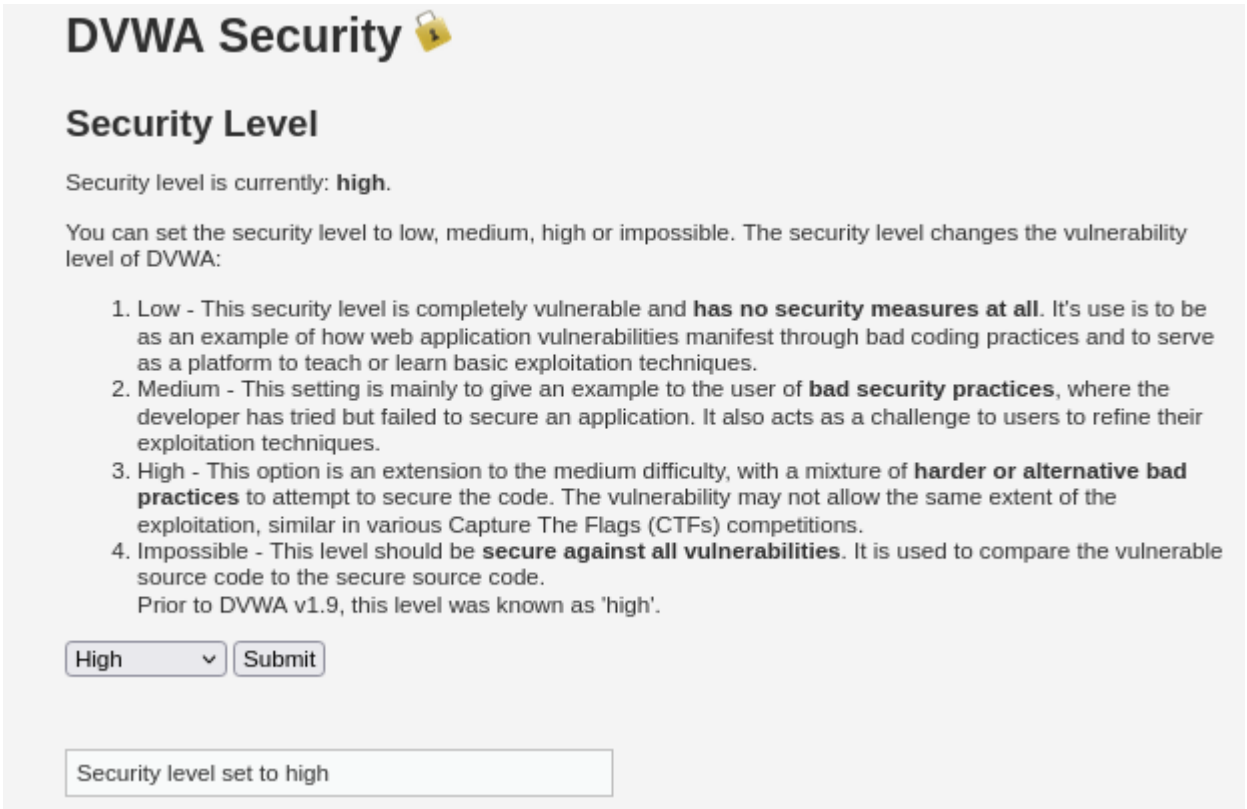
Request Response

Pretty Raw Hex \n

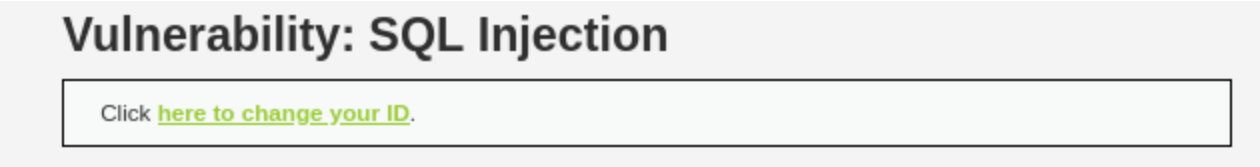
```
POST /vulnerabilities/sqli/ HTTP/1.1
Host: 127.0.0.1:42001
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 18
Origin: http://127.0.0.1:42001
Connection: keep-alive
Referer: http://127.0.0.1:42001/vulnerabilities/sqli/
Cookie: PHPSESSID=5fd9adb51786e5c4ccd9972a6c016c4c; security=medium
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
DNT: 1
Sec-GPC: 1
Priority: u=0, i

id=1&Submit=Submit
```

- **High Security:**

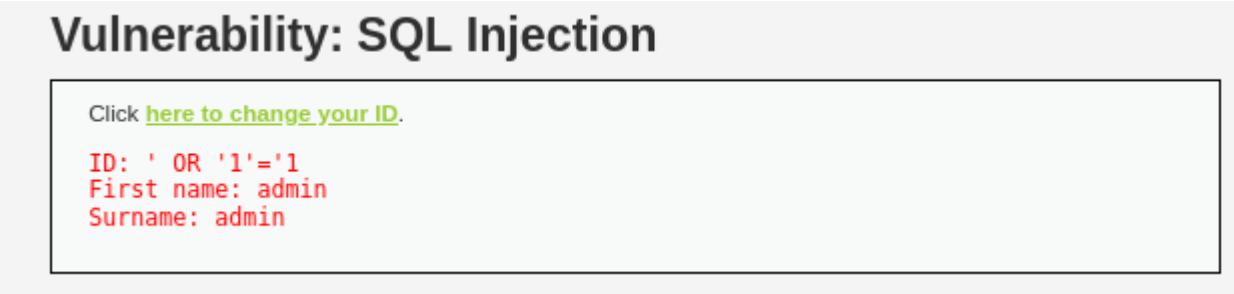
  → Changed Security Level to **High.**



Result:



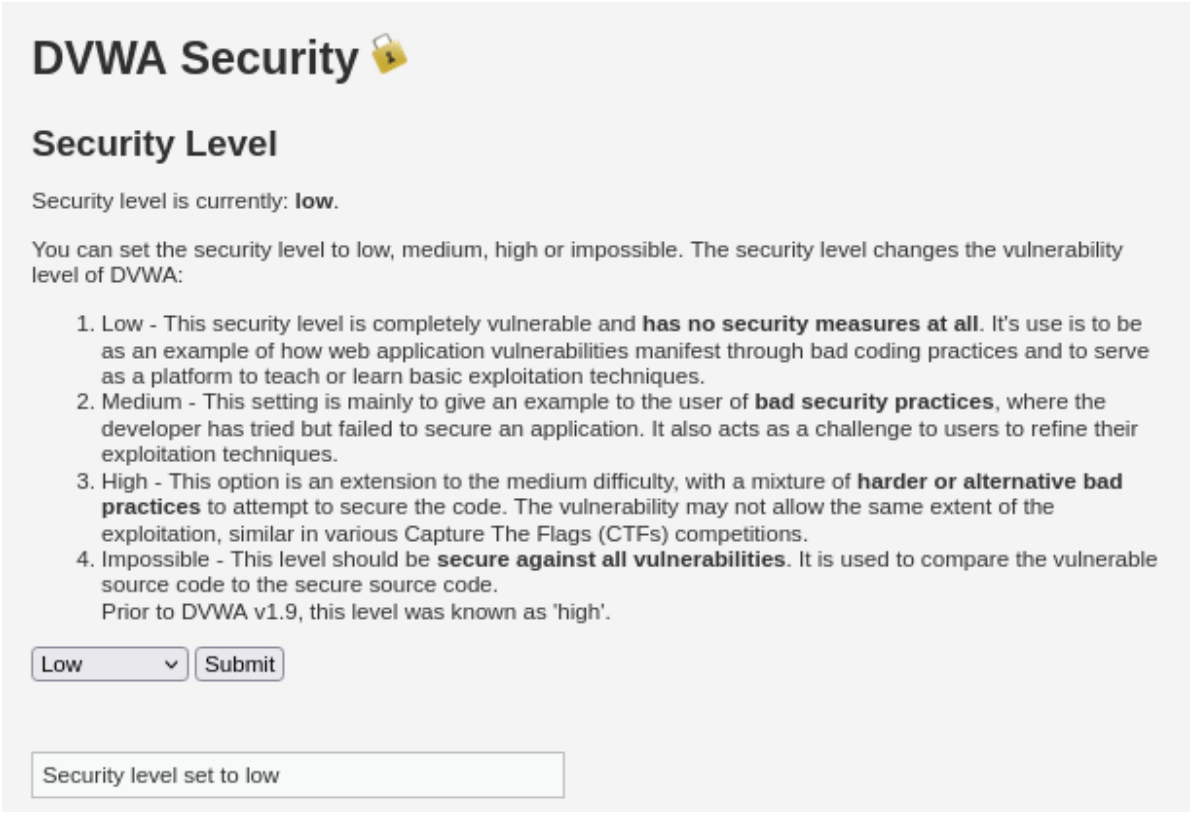
- **Went inside the link.**
- **Payload Used:** ‘ OR ‘1’=’1



2. CROSS-SITE SCRIPTING (Reflected XSS)

- **Low Security**

  → Set security level to **Low**



→ XSS(Reflected) Page → Entered the payload.

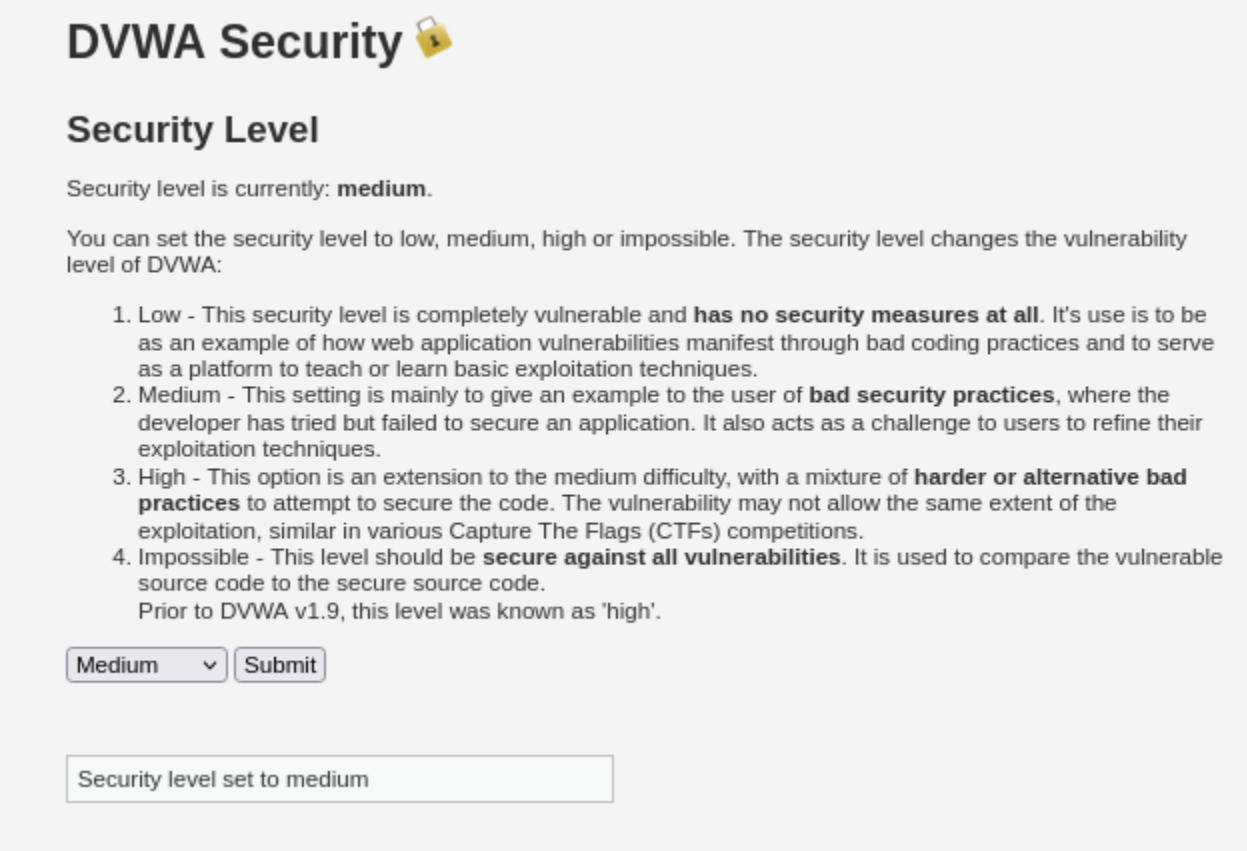
Payload:

<script>alert(‘XSS’)</script>

Result:

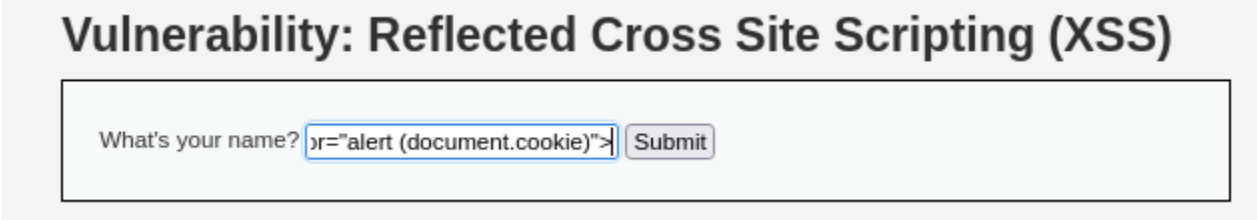
- **Medium Security**

  → Set Security Level to **Medium.**

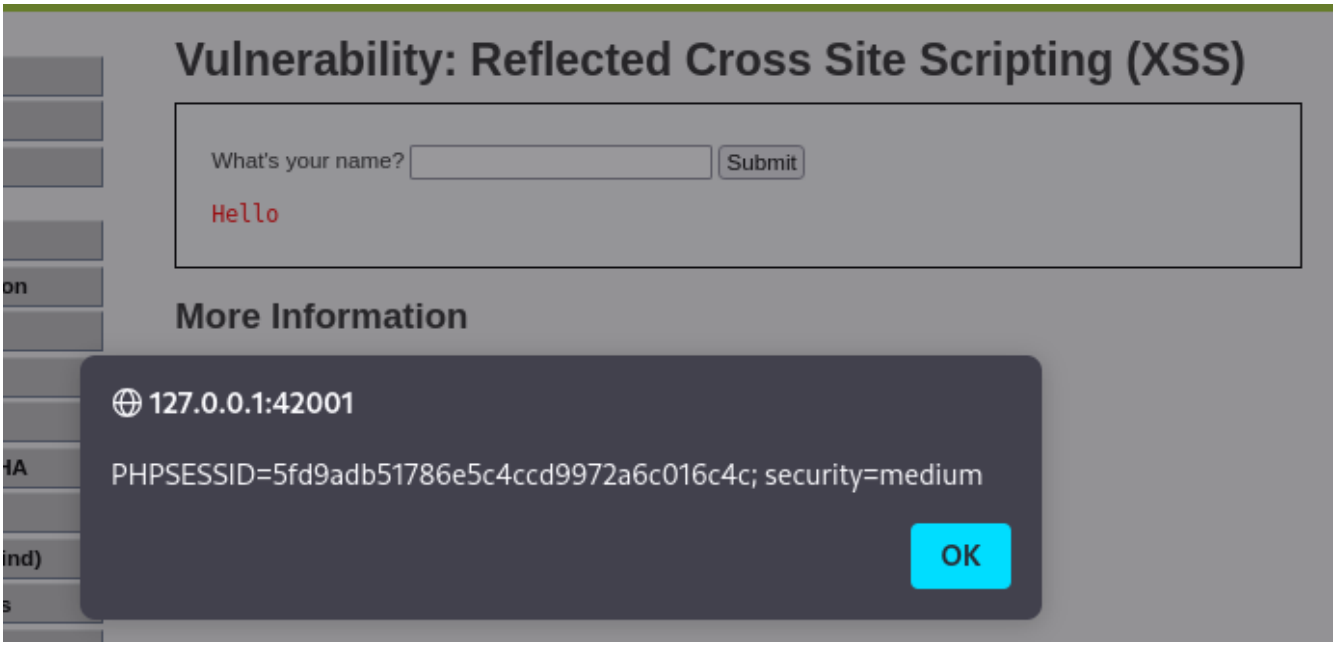


Payload Used:

<img src=x onerror=alert(1)>

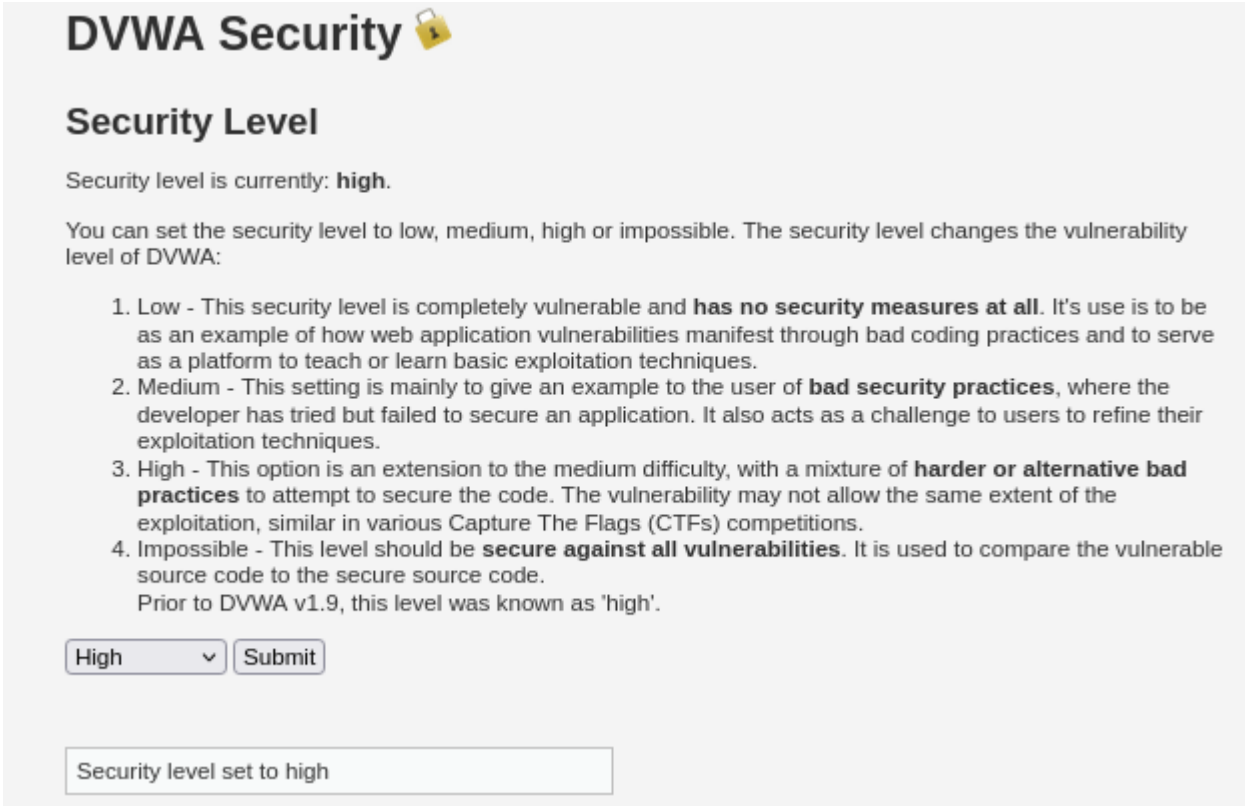


Result:



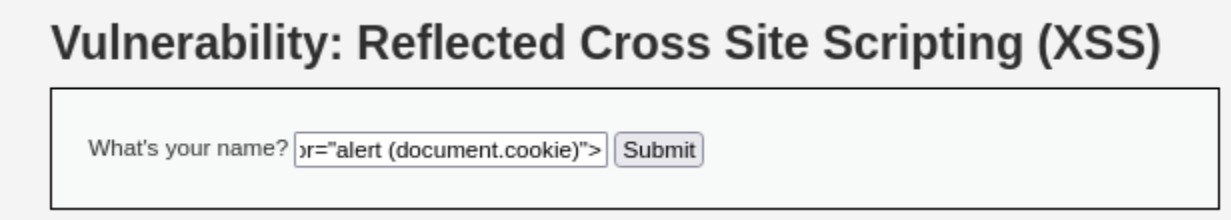
- **High Security:**

  → Changed security level to **High.**

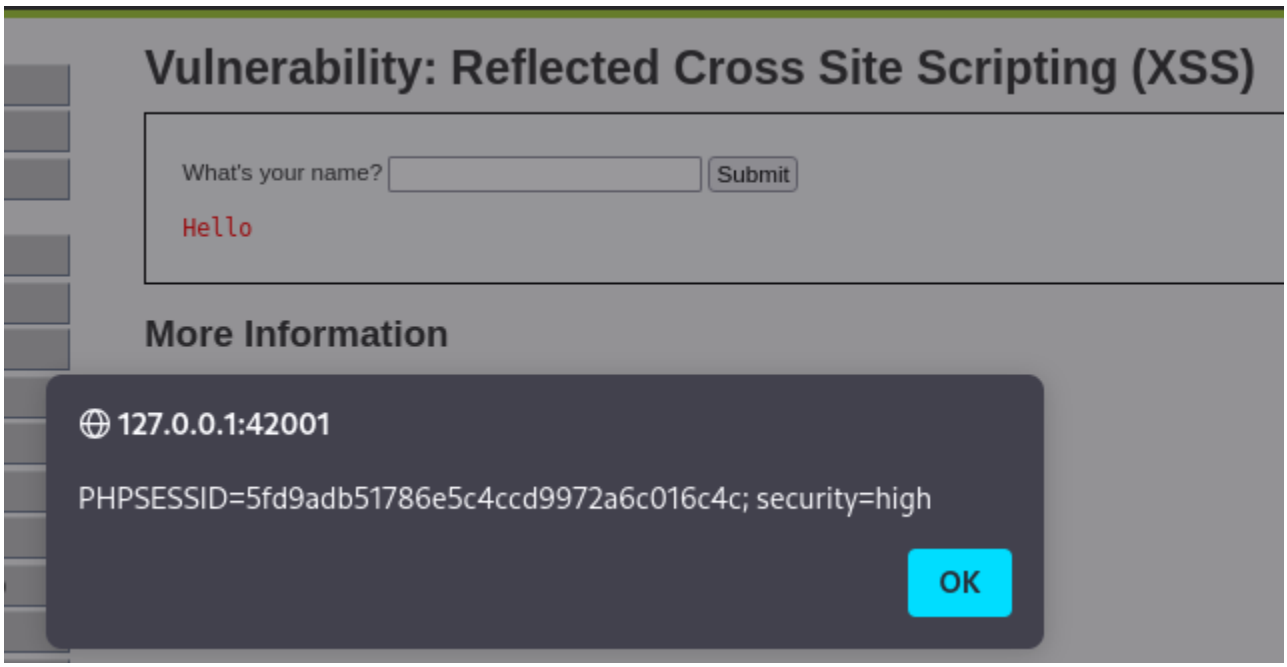


Payload:

<img src=x onerror=alert(1)>



Result:



Comparison:

SQL Injection

- **Low** – Direct SQLi works with ‘OR ‘1’=’1
- **Medium** – Input field removed. Was only able to get the users name one by one.
- **High** – Parametered queries /stronger input validation prevents SQLi by a higher level.

XSS (Reflected)

- **Low** - <script> payload executes directly.
- **Medium** - <script> filtered. Bypass possible with <img onerror>.
- **High** – Stricter filtering / encoding stops direct scripts but some HTML payloads may still fire.

Observation:

- **Medium Security:** Basic input filtering or restricting user input mechanisms (drop-down instead of free test).
- **High Security:** Parameterized queries, server-side validation, and stronger sanitation/encoding to neutralize malicious input.

Conclusion:

DVWA progressively implements stronger defenses as the security level increases. A **Low,** vulnerabilities are fully exploitable. At **Medium ,** superficial defenses like input restrictions and script filtering make attacks harder but not impossible. At **High,** parameterized queries and stricter sensitization significantly reduce exploitable vulnerabilities. This exercise demonstrates how incremental defensive measures impact common web application attacks.